

CalcApp

Agile & DevOps in Practice — Final Assessment

Josiane Ishimwe

Data Engineering

8 July 2026

1. Product Vision

For anyone who needs quick calculations, CalcApp is a simple command-line calculator that performs basic arithmetic operations reliably and accurately.

2. Product Backlog & User Stories

Story 1 — Addition

As a user, I want to add two numbers, so that I can get their sum.

Estimate: 2 points | Priority: HIGH

Acceptance Criteria:

- User can enter two numbers
- App returns correct sum
- Works with decimals
- Works with negative numbers
- Result displays clearly (e.g. $2 + 3 = 5$)

Story 2 — Subtraction

As a user, I want to subtract two numbers, so that I can find the difference.

Estimate: 2 points | Priority: HIGH

Acceptance Criteria:

- User can enter two numbers
- App returns correct difference
- Works with decimals
- Works with negative numbers
- Result displays clearly (e.g. $5 - 3 = 2$)

Story 3 — Multiplication

As a user, I want to multiply two numbers, so that I can get the product.

Estimate: 2 points | Priority: HIGH

Acceptance Criteria:

- User can enter two numbers
- App returns correct product
- Works with decimals
- Works with negative numbers
- Result displays clearly (e.g. $3 * 4 = 12$)

Story 4 — Division

As a user, I want to divide two numbers, so that I can get the result.

Estimate: 3 points | Priority: HIGH

Acceptance Criteria:

- User can enter two numbers
- App returns correct result
- Works with decimals
- Works with negative numbers
- Result displays clearly (e.g. $10 / 2 = 5$)

Story 5 — Division by Zero Error

As a user, I want to see an error when dividing by zero, so that the app doesn't crash.

Estimate: 3 points | Priority: MEDIUM

Acceptance Criteria:

- App does NOT crash when dividing by zero
- Clear error message shown
- App continues running after error
- All other operations still work

Story 6 — Calculation History

As a user, I want to see a history of my calculations, so that I can review past results.

Estimate: 4 points | Priority: LOW

Acceptance Criteria:

- Every calculation saved automatically
- User types 'history' to see all past calculations
- History shows in order from first to latest
- Each entry shows full calculation

3. Acceptance Criteria

Each user story's detailed acceptance criteria are listed above under Section 2 (Product Backlog & User Stories). At a summary level, the CalcApp prototype is considered acceptable when:

- All four arithmetic operations (add, subtract, multiply, divide) return mathematically correct results, including with decimals and negative numbers.
- The application never crashes on invalid input, division by zero and unknown operators are handled gracefully with clear error messages.
- Every calculation is automatically stored and retrievable via the 'history' command, in chronological order.

- All results are displayed in a clear, human-readable format.

4. Definition of Done

Every backlog story must meet all of the following before it counts as complete:

- ☑ Code written and working locally
- ☑ All acceptance criteria met
- ☑ At least one unit test written
- ☑ All tests passing
- ☑ CI pipeline passing on GitHub
- ☑ Code committed with a clear, descriptive message
- ☑ No crashes or unhandled errors

5. Sprint 0 Plan

Setup Checklist

- ☑ GitHub repository created
- ☑ Project folder structure set up
- ☑ README written with product vision
- ☑ Product backlog documented
- ☑ Acceptance criteria written
- ☑ Definition of Done defined
- ☑ CI pipeline configured
- ☑ Sprint 1 planned

Sprint 1 Plan

Goal: Deliver basic arithmetic operations

Story	Points
Story 1: Addition	2
Story 2: Subtraction	2
Story 3: Multiplication	2
Total	6

Sprint 2 Plan

Goal: Complete calculator with error handling and history

Story	Points
Story 4: Division	3
Story 5: Division by zero error	3
Story 6: Calculation history	4

Story	Points
Total	10

6. Sprint 1 Review

Date: 08 July 2026 | **Developer:** Josiane Ishimwe

Sprint Goal

Deliver basic arithmetic operations (addition, subtraction, multiplication) with passing tests and a working CI pipeline.

Planned vs Delivered

Story	Points	Status
Story 1: Addition	2	Completed
Story 2: Subtraction	2	Completed
Story 3: Multiplication	2	Completed

Stories planned: 3 | Stories delivered: 3 | Points planned: 6 | Points delivered: 6

Definition of Done Checklist

- ☒ Code written and working locally
- ☒ All acceptance criteria met
- ☒ Unit tests written for each story
- ☒ All tests passing (21/21)
- ☒ CI pipeline passing on GitHub
- ☒ Code committed with clear messages
- ☒ No crashes or unhandled errors

Demo Evidence

Addition

```
Enter calculation: 3 + 2
2026-07-08 10:57:15 - INFO - Addition: 3.0 + 2.0 = 5.0
Result: 3.0 + 2.0 = 5.0
```

Subtraction

```
Enter calculation: 6 - 2
2026-07-08 10:59:10 - INFO - Subtraction: 6.0 - 2.0 = 4.0
Result: 6.0 - 2.0 = 4.0
```

Multiplication

```
Enter calculation: 3 * 4
2026-07-08 - INFO - Multiplication: 3.0 * 4.0 = 12.0
Result: 3.0 * 4.0 = 12.0
```

Tests Passing

What Was Not Delivered

All planned Sprint 1 stories were delivered successfully.

Backlog Remaining for Sprint 2

- Story 4: Division (3 points)
- Story 5: Division by zero error (3 points)
- Story 6: Calculation history (4 points)

7. Sprint 1 Retrospective

Date: 08 July 2026 | Developer: Josiane Ishimwe

What Went Well

- CI pipeline was set up successfully and triggered automatically on every push
- All 3 Sprint 1 stories were delivered within the sprint
- All 21 unit tests passed on first full run
- Logging was implemented and working from the start
- Commit history shows clear incremental progress

What Didn't Go Well

- Early commits triggered red CI checks because code was incomplete at the time of pushing — should have ensured at least basic tests existed before first push
- Some commits were grouped together rather than being split per feature — for example, Sprint 1 and Sprint 2 documentation was committed together in one message

What Puzzled Me

- Understanding why the CI pipeline showed red on early commits even though the final result was correct
- The difference between committing empty files vs files with content and how that affects the pipeline

Improvements for Sprint 2

Improvement 1:

What went wrong: Some commits combined multiple changes

How I will fix it: Commit immediately after each individual function works — one commit per story feature

Improvement 2:

What went wrong: CI showed red on early pushes because files were empty

How I will fix it: Only push code files after they have at least basic content and tests written

Estimation Accuracy

Story	Estimated	Felt
Story 1: Addition	2 pts	About right
Story 2: Subtraction	2 pts	About right
Story 3: Multiplication	2 pts	About right

Lesson for Sprint 2: Estimates were accurate for simple operations. Division and error handling may need slightly more buffer since they involve edge cases.

8. Sprint 2 Review

Date: 08 July 2026 | Developer: Josiane Ishimwe

Sprint Goal

Complete the calculator with division, error handling, calculation history, and logging. Apply improvements from Sprint 1 retrospective.

Sprint 1 Improvements Applied

Improvement 1: Commit per feature not per session

Applied: Yes, Evidence: Separate commits for calculator structure, tests, and documentation visible in commit history

Improvement 2: Only push files with content

Applied: Yes, Evidence: Final commits show green CI pipeline

Planned vs Delivered

Story	Points	Status
Story 4: Division	3	Completed
Story 5: Division by zero error	3	Completed
Story 6: Calculation history	4	Completed

Stories planned: 3 | Stories delivered: 3 | Points planned: 10 | Points delivered: 10

Definition of Done Checklist

- ☒ Code written and working locally
- ☒ All acceptance criteria met
- ☒ Unit tests written for each story
- ☒ All Sprint 1 and Sprint 2 tests passing (21/21)
- ☒ CI pipeline passing on GitHub
- ☒ Logging added to all operations
- ☒ Error handling working correctly

- ☑ Code committed with clear messages
- ☑ No crashes or unhandled errors

Monitoring and Logging Evidence

Logging captures every operation automatically:

```
2026-07-08 10:57:15,968 - INFO - Addition: 3.0 + 2.0 = 5.0
2026-07-08 10:59:10,340 - INFO - Subtraction: 6.0 - 2.0 = 4.0
2026-07-08 10:59:37,781 - ERROR - Division by zero attempted
2026-07-08 10:59:46,797 - INFO - Division: 5.0 / 2.0 = 2.5
```

- INFO logs record every successful calculation
- ERROR logs capture failed operations (e.g. divide by zero)
- Timestamps on every log entry for full traceability

Demo Evidence

Division

```
Enter calculation: 5 / 2
2026-07-08 - INFO - Division: 5.0 / 2.0 = 2.5
Result: 5.0 / 2.0 = 2.5
```

Division by Zero Error

```
Enter calculation: 5 / 0
2026-07-08 - ERROR - Division by zero attempted
Error: Cannot divide by zero
```

App Continues After Error

```
Enter calculation: 3 * 4
Result: 3.0 * 4.0 = 12.0
```

Calculation History

```
Enter calculation: history
--- Calculation History ---
3.0 + 2.0 = 5.0
6.0 - 2.0 = 4.0
5.0 / 2.0 = 2.5
3.0 * 4.0 = 12.0
```

Unknown Operator Handled

```
Enter calculation: 5 : 0
Unknown operator ':'. Use +, -, *, /
```

Full Project Summary

Metric	Result
Total stories delivered	6/6
Total story points delivered	16/16
All tests passing	21/21
CI pipeline working	Green
Logging implemented	INFO + ERROR
Error handling working	Yes

Metric	Result
History feature working	Yes

9. Sprint 2 Retrospective

Date: 08 July 2026 | **Developer:** Josiane Ishimwe

What Went Well

- All 6 stories delivered across both sprints
- 21/21 tests passing with no failures
- CI pipeline running automatically and showing green
- Logging captured both successful operations and errors
- Error handling prevented app from crashing on invalid input like division by zero and unknown operators
- History feature worked correctly showing all calculations in order

What Didn't Go Well

- Sprint 1 and Sprint 2 were planned and partly executed on the same day, which made the sprint separation less clear than it should be in real practice
- Some documentation commits combined Sprint 1 and Sprint 2 content instead of keeping them separate

Did Sprint 1 Improvements Help?

Improvement 1: Commit per feature

Yes, later commits were more focused and descriptive, making the history easier to follow

Improvement 2: Only push files with content

Yes, the final commits show green CI instead of red, confirming that pushing complete code makes the pipeline more reliable

Overall Project Lessons

About Agile:

Working in sprints forces you to think about priority. Knowing Sprint 1 only covers addition, subtraction, and multiplication meant I could focus completely without being distracted by division or history features.

About DevOps:

Setting up CI early was the right decision. Seeing the red crosses on early commits actually helped me understand what the pipeline was checking — it made the green tick on the final commit feel meaningful.

About estimating:

The simple operations (add, subtract, multiply) were accurately estimated at 2 points each. Division with error handling deserved its 3 points because edge cases like divide by zero required extra thought.

About myself:

I work better when I break tasks into very small steps. The commit-per-feature approach kept me focused and made progress visible even when the full feature wasn't done yet.

What I Would Do Differently

- Start with tests written BEFORE the code (TDD — Test Driven Development) so CI is green from the start
- Keep Sprint 1 and Sprint 2 on separate days to make the timeline clearer
- Add more edge case tests (e.g. very large numbers, floating point precision)

Final Self Assessment

Area	Score
Agile practice quality	4/5
DevOps implementation	4/5
Commit discipline	3/5
Test coverage	5/5
Reflection depth	4/5

Most proud of: 21/21 tests passing and the logging system working automatically on every operation

Will improve next project: Keeping sprints on separate days and committing more granularly per individual function

10. CI/CD Evidence

GitHub Actions workflow configuration (.github/workflows/ci.yml):

```
name: CalcApp CI Pipeline

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.9'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
        run: pytest test_calculator.py -v
```

Commits		
master	All users	All time
Commits on Jul 8, 2026		
Implement division functionality and enhance calculation history management	8937291	
Joh-Ishimwe committed 2 hours ago · 1 / 1		
Sprint 2 Retrospective	42c2837	
Joh-Ishimwe committed 2 hours ago · 1 / 1		
Sprint 2 Review document	d25a5c8	
Joh-Ishimwe committed 2 hours ago · 1 / 1		
add srpint 2 features and test codes	5934421	
Joh-Ishimwe committed 2 hours ago · 1 / 1		
Sprint 1 Retrospective document	9c8ca40	
Joh-Ishimwe committed 2 hours ago · 1 / 1		
add Sprint 1 review document	2a25373	
Joh-Ishimwe committed 2 hours ago · 1 / 1		
add sprint 2 features and test codes	e379477	
Joh-Ishimwe committed 2 hours ago · 1 / 1		
add unit tests for Sprint 1 stories	7fdffa8	
Joh-Ishimwe committed 2 hours ago · 1 / 1		
add basic calculator structure and logging	4a0c433	
Joh-Ishimwe committed 2 hours ago · 0 / 1		

The GitHub Actions run history

Project Structure

```

calcapp/
├── .github/
│   └── workflows/
│       └── ci.yml
├── docs/
│   ├── backlog.md
│   ├── sprint0-plan.md
│   ├── sprint1-review.md
│   ├── sprint1-retro.md
│   ├── sprint2-review.md
│   └── sprint2-retro.md
├── calculator.py
├── test_calculator.py
├── requirements.txt
└── README.md

```

11. Testing Evidence

Unit and integration tests (test_calculator.py) — 21 tests covering Sprint 1 and Sprint 2 stories:

Sprint 1 Tests — Addition, Subtraction, Multiplication

```

class TestAddition:
    def test_add_two_positive_numbers(self):
        assert add(2, 3) == 5
    def test_add_with_zero(self):
        assert add(0, 5) == 5
    def test_add_negative_numbers(self):
        assert add(-2, -3) == -5

```

```
def test_add_decimals(self):
    assert add(1.5, 2.5) == 4.0
```

Sprint 2 Tests — Division & History

```
class TestDivision:
    def test_divide_two_positive_numbers(self):
        assert divide(10, 2) == 5
    def test_divide_by_zero_raises_error(self):
        with pytest.raises(ValueError) as error:
            divide(10, 0)
        assert "Cannot divide by zero" in str(error.value)

class TestHistory:
    def test_history_preserves_order(self):
        save_to_history("first")
        save_to_history("second")
        assert get_history()[0] == "first"
        assert get_history()[1] == "second"
```

Test Run Result

```
===== 21 passed in 0.09s =====
```

```
PS C:\Users\jishi\Desktop\Amalitech\CalcApp_Agile_DevOps_Assessment> pytest test_calculator.py -v
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\jishi\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\jishi\Desktop\Amalitech\CalcApp_Agile_DevOps_Assessment
collected 21 items

test_calculator.py::TestAddition::test_add_two_positive_numbers PASSED [ 4%]
test_calculator.py::TestAddition::test_add_with_zero PASSED [ 9%]
test_calculator.py::TestAddition::test_add_negative_numbers PASSED [ 14%]
test_calculator.py::TestAddition::test_add_decimals PASSED [ 19%]
test_calculator.py::TestSubtraction::test_subtract_two_positive_numbers PASSED [ 23%]
test_calculator.py::TestSubtraction::test_subtract_gives_negative PASSED [ 28%]
test_calculator.py::TestSubtraction::test_subtract_with_zero PASSED [ 33%]
test_calculator.py::TestSubtraction::test_subtract_decimals PASSED [ 38%]
test_calculator.py::TestMultiplication::test_multiply_two_positive_numbers PASSED [ 42%]
test_calculator.py::TestMultiplication::test_multiply_by_zero PASSED [ 47%]
test_calculator.py::TestMultiplication::test_multiply_negative_numbers PASSED [ 52%]
test_calculator.py::TestMultiplication::test_multiply_decimals PASSED [ 57%]
test_calculator.py::TestDivision::test_divide_two_positive_numbers PASSED [ 61%]
test_calculator.py::TestDivision::test_divide_gives_decimal PASSED [ 66%]
test_calculator.py::TestDivision::test_divide_negative_numbers PASSED [ 71%]
test_calculator.py::TestDivision::test_divide_by_zero_raises_error PASSED [ 76%]
test_calculator.py::TestDivision::test_divide_by_zero_does_not_crash PASSED [ 80%]
test_calculator.py::TestHistory::test_history_starts_empty PASSED [ 85%]
test_calculator.py::TestHistory::test_save_to_history PASSED [ 90%]
test_calculator.py::TestHistory::test_history_saves_multiple_entries PASSED [ 95%]
test_calculator.py::TestHistory::test_history_preserves_order PASSED [100%]

===== 21 passed in 0.10s =====
```

Terminal screenshot of the full pytest run